© Victor Tan 2025

Angular Workshop
Lab 4: Reactive Forms
Exercise Solutions

1 LAB SETUP 1

2 GENERATING A NEW ANGULAR PROJECT 1

3 ADDING A BASIC FORM CONTROL USING FORMCONTROL 1

4 USING FORMGROUP TO GROUP RELATED FORM CONTROLS 2

5 SUBMITTING THE FORM AND ACCESSING FORM AND FORM CONTROL VALUES 3

6 UPDATING ALL OR A PORTION OF A FORMGROUP 4

7 CONTROL STATUS FOR A FORMCONTROL AND FORMGROUP 6

8 VALIDATING FORM CONTROLS 6

9 DISPLAY VALIDATION ERROR MESSAGES 6

10 USING FORMBUILDER AS A SHORTCUT FOR CREATING FORMGROUP 7

11 USING FORMARRAY TO BUILD DYNAMIC FORMS 7

12 ADDING OTHER FORM CONTROL ELEMENTS (RADIO BUTTONS, CHECKBOXES, ETC) TO A REACTIVE FORM 7

1 Lab setup

2 Generating a new Angular project

3 Adding a basic form control using FormControl

Q1. Create a radio button with a random value and bind it to a FormControl instance property in the component

Q2: Use interpolation to display the value of the radio button in the template itself.

app.component.ts

```
// Q1
language = new FormControl('');
```

app.component.html

```
<!-- Q1 -->
<input type="radio" id="html" [formControl]="language" value="HTML">
<label for="html">HTML</label><br>

<!-- Q2 -->

<p>The value of the radio button is : {{ language.value }}</p>
```

4 Using FormGroup to group related form controls

Q1. In the template, add in a new numeric input field for age just after the email field in the main FormGroup. Bind this element to a new FormControl instance in the component.

Q2. In the template, add in a new text input field for zipcode just after the state field in the nested FormGroup. Bind this element to a new FormControl instance in the component.

Q3: Use interpolation to display the values of these two new fields in the template itself.

app.component.ts

```
profileForm = new FormGroup({
  firstname: new FormControl(''),
  lastname: new FormControl(''),
  email: new FormControl(''),
  // Q1
  age: new FormControl(0),
  address: new FormGroup({
    street: new FormControl(''),
    city: new FormControl(''),
    state: new FormControl(''),
```

```
      // Q2
      zipcode: new FormControl('')
    })
  });
```

app.component.html

```
  <!-- Q1 -->
  <p>
    <label for="age">Age :</label>
    <input type="number" id="age" formControlName="age">
  </p>

    <!-- Q2 -->
    <p>
      <label for="zipcode">Zip Code</label>
      <input type="text" id="zipcode" formControlName="zipcode">
    </p>



<!-- Q3 -->

<p>The age is : {{ profileForm.value?.age }}</p>

<p>The zipcode in the address is :  {{ profileForm.value.address?.zipcode
}}</p>
```

5 Submitting the form and accessing form and form control values

Q1: Repeat Q1 and Q2 from the previous section. Add in statements in the component method that handles the submitted form data in order to obtain the value for the age and zipcode form elements.

app.component.ts

```
    // Q1
    console.log('The age is ', this.profileForm.value.age);
```

```
    console.log('The zipcode is ', this.profileForm.value.address?.zipcode);
```

6 Updating all or a portion of a FormGroup

Q1: Repeat Q1 and Q2 from the Section 4. Add in additional logic to the component method that uses setValue in order to provide new values for the zipcode and age controls.

Q2: Add in additional logic to the component method that uses patchValue in order to provide new values for the zipcode and age controls.

app.component.ts

```
  profileForm = new FormGroup({
    firstname: new FormControl(''),
    lastname: new FormControl(''),
    email: new FormControl(''),
    // Q1
    age: new FormControl(0),
    address: new FormGroup({
      street: new FormControl(''),
      city: new FormControl(''),
      state: new FormControl(''),
      // Q1
      zipcode: new FormControl('')
    })
  });


  // Q1
  setContentForEntireForm() {
    // Create an object with properties
    // that match the structure of the form exactly

    let supermanProfile = {
      firstname : 'Clark',
      lastname: 'Kent',
      email: 'superman@gmail.com',
      age: 44,
      address : {
        street : '99 Louis Lane',
        city : 'Metropolis',
        state : 'New York',
```

```
        zipcode: '9999',
      }

    };

    this.profileForm.setValue(supermanProfile);

  }


  // Q2
  setContentForPartOfForm() {

    let updatedProfile = {
      firstname : 'Tony',
      lastname: 'Stark',
      age : 56,
      address : {
        street : '100 Stark Industries',
        zipcode : '4444',
      }

    };

    this.profileForm.patchValue(updatedProfile);

  }
```

app.component.html

```
<!-- Q1 -->
<p>
  <label for="age">Age :</label>
  <input type="number" id="age" formControlName="age">
</p>

  <!-- Q1 -->
  <p>
    <label for="zipcode">Zip Code</label>
    <input type="text" id="zipcode" formControlName="zipcode">
  </p>
```

7 Control status for a FormControl and FormGroup

No exercises here

8 Validating form controls

Q1: Add an additional validator to the first name field so that it only contains letters (no numbers, spaces or any other characters). Hint: Angular - Validators

app.component.ts

```
    // Q1
    firstname: new
FormControl('',[Validators.required,Validators.minLength(3),
Validators.pattern('[a-zA-Z]*')]),
```

For more info on regular expressions in JavaScript and how to use them:
https://www.javascripttutorial.net/javascript-regular-expression/
https://www.w3schools.com/jsref/jsref_obj_regexp.asp

9 Display validation error messages

Q1: Add three validators to the job field:

- Make it required
- Make it have a maximum of 10 characters
- Make it only contain letters (no numbers, spaces or any other characters). Hint: Angular - Validators

Q2: Add appropriate conditional logic to display validation error messages for the job field.

app.component.ts

```
    // Q1
```

```
    job: new
FormControl('',[Validators.required,Validators.maxLength(10),Validators.patt
ern('[a-zA-Z]*')])
```

app.component.html

```
  <!-- Q2: Validation error message for job -->
  <div *ngIf='job?.invalid && (job?.dirty || job?.touched)'
style="color:red">
    <div *ngIf="job?.errors?.['required']">
      Job is required.
    </div>
    <div *ngIf="job?.errors?.['maxlength']">
      Job cannot be more than 10 characters long.
    </div>
    <div *ngIf="job?.errors?.['pattern']">
      Job can only contain alphabets
    </div>
  </div>
```

10 Using FormBuilder as a shortcut for creating FormGroup

No exercises here

11 Using FormArray to build dynamic forms

No exercises here

12 Adding other form control elements (radio buttons, checkboxes, etc) to a reactive form

Q1. Create an array of strings in the component class.

```
countries = ['USA', 'UK', 'China', 'India'];
```

Use these string items as options in a drop down list in the template via a @for directive similar to the case of the radio buttons. The drop down list (the <select> element) is then bound to a suitable Form Control in the FormBuilder group in the component.

app.component.ts

```
// Array of strings for drop down list choices
countries = ['USA', 'UK', 'China', 'India'];

constructor(private fb: FormBuilder) {
  this.profileForm = this.fb.group({
    favlanguage: [''],
    favcountry: [this.countries[0]],
    favcars : this.fb.array([])
  });

}
```

app.component.html

```
<!-- Creating Drop down list options -->
<p>Select your favourite country :</p>
<select formControlName="favcountry">
  @for (country of countries; track country) {
    <option [value]="country">
      {{country}}
    </option>
  }
</select>
```